

Final Project

End-to-End Data Science Pipeline with Interactive Dashboard

Course: Data Science and Analytics (MCT-341L) Instructor: Dr. Muhammad Ahsan Naeem

Covers: Labs 1–13 + Clustering (Upcoming Lab)

Submit ONE ZIP archive per group, named exactly as:

FinalProject-2023-MC-XX_MC-YY.zip

where XX is the **lower** and YY is the **higher** roll number in your group

(e.g. FinalProject-2023-MC-22_MC-28)

One submission per group — do not submit individually. Wrong archive name or internal structure = submission not accepted. See the Deliverables section for the required folder layout.

Course Learning Outcomes (CLOs) Covered in This Project

CLO 1	Apply statistical and computational tools to real-world problems	PLO 3
CLO 2	Build and assess various data-based models	PLO 2
CLO 3	Effectively communicate data analytics techniques and results	PLO 10
CLO 4	Create and deploy an interactive solution including dashboards	PLO 3

Project Overview

The final project integrates every major skill from this course into a single coherent data science workflow. You will select a large, publicly available dataset relevant to engineering or science, apply the complete pipeline from raw data to insight, build and evaluate predictive and unsupervised models, and deploy an interactive **Streamlit** dashboard that any non-technical user can navigate.

This project mirrors the workflow of a professional data scientist: you start with messy real-world data, make principled modelling decisions, and present results clearly. **Dataset selection is your first and most consequential decision** — choose a dataset that is large enough, messy enough, and domain-relevant enough to reward serious effort. A trivially clean toy dataset will not earn high marks even if every code cell runs.

AI-Use Policy

AI tools may be used for syntax assistance, debugging, documentation lookup, and boilerplate code generation. However, all interpretations, explanations, captions, conclusions, and decision justifications submitted in the notebook **must be written and understood by the students themselves**. Both partners must be able to explain every line of code and every conclusion during the viva. Submitting AI-generated analysis text as your group's

own is academic dishonesty.

Submission & Viva

There is **one final submission** for this project — a single ZIP archive uploaded to LMS by the deadline announced in class.

Note: The in-lab **viva and dashboard demo** takes place in the final lab session. Both group members must be present with their dashboard running locally. Inability to demo the dashboard on the day counts as a non-submission for Phase 9.

Dataset Selection

Your dataset must satisfy every criterion below. No approval is needed — use this list as your own checklist before you begin.

- **Engineering or applied-science domain** (mandatory). The dataset must come from a field such as: manufacturing, power systems, robotics, aerospace, civil/environmental engineering, biomedical instrumentation, transportation, energy, telecommunications, or industrial process control. Purely social-science, economic, or entertainment datasets do not qualify.
- Approximately 10,000 rows and ≥ 8 feature columns
- At least 3 numerical features and 2 categorical features
- Contains a suitable continuous target variable (for regression) *and* a suitable categorical target (for classification); these may be the same column treated differently (e.g. a continuous sensor reading and a binned fault-severity label) or two separate columns. If no natural classification target exists, you may derive one using domain-informed thresholds (e.g. binning a sensor reading into fault/normal based on a manufacturer's specification limit) — but arbitrary quantile-based splits are not acceptable.
- Publicly available under an open licence (UCI, Kaggle, Creative Commons, Open Government, etc.)
- **Not pre-cleaned:** the dataset must require genuine preprocessing effort (missing values, outliers, or encoding decisions). A dataset with zero missing values and no anomalies provides insufficient challenge for this project.

Recommended dataset sources:

UCI ML Repository	https://archive.ics.uci.edu/
Kaggle Datasets	https://www.kaggle.com/datasets
Pakistan Bureau of Stats	https://www.pbs.gov.pk/
World Bank Open Data	https://data.worldbank.org/
NASA Open Data	https://data.nasa.gov/
OpenAQ (air quality)	https://openaq.org/ — includes Lahore, Karachi
Our World in Data	https://ourworldindata.org/

Example datasets suitable for this project:

1. **AI4I 2020 Predictive Maintenance** (UCI, 10 000 rows) — machine failure classification and tool-wear regression; ideal for mechatronics students. <https://archive.ics.uci.edu/dataset/601>
2. **Smart Grid Stability** (UCI, 60 000 rows) — 12 electrical features, binary grid-stability classification. <https://archive.ics.uci.edu/dataset/471>
3. **Combined Cycle Power Plant** (UCI, 9 568 rows) — predict hourly electrical output from ambient conditions (temperature, pressure, humidity, exhaust vacuum). <https://archive.ics.uci.edu/dataset/294>

4. **Global Air Quality Dataset** (Kaggle) — PM2.5, AQI, and multiple pollutants for cities worldwide including Lahore and Karachi. Search Kaggle for “global air quality data”.
5. **Road Safety / Traffic Accidents** (Kaggle / UK DfT) — accident severity as classification target; temporal, spatial, and vehicle features for regression on injury count.

Note: If you are unsure whether your chosen dataset qualifies, ask yourself: “*Could this dataset appear in an IEEE Transactions or engineering conference paper?*” If yes, it almost certainly qualifies. If no (e.g. movie reviews, sports scores, social-media engagement), choose a different dataset.

Phase 1: Dataset Acquisition & Initial Exploration | CLO 1 | 2 marks

Load your dataset and establish a complete picture of its structure before making any changes. Every discovery in this phase informs every decision in every later phase.

- 1.1. **Load and inspect.** Use `pd.read_csv()` or the appropriate loader for your file format. Display the first 10 rows, `df.info()`, `df.shape`, and `df.memory_usage(deep=True).sum()`. Print the exact column names and their inferred data types.
- 1.2. **Data card.** In a markdown cell, write a structured data provenance record covering: source URL and licence, data collection method (survey, sensor, simulation, scrape), approximate collection period, known limitations, and any sampling biases documented by the dataset creator. This is your permanent record of where the data came from and what it represents.
- 1.3. **Statistical snapshot.** Run `df.describe(include="all")`. In a following markdown cell, interpret each column’s summary: flag any numeric column where mean and median differ by more than 20% (a sign of skewness that will affect modelling), and any column whose maximum or minimum seems physically implausible (potential data-entry error or unit mismatch).
- 1.4. **Target variable definition.** In a clearly labelled markdown cell, define and justify your two modelling targets:
 - *Regression target* — must be continuous; state its physical meaning and units (e.g. “Electrical power output in MW”).
 - *Classification target* — must be categorical; if it is derived by binning a continuous column, explain and justify the threshold values with domain reasoning (not arbitrary quantile cuts).
- 1.5. **Class balance check.** For the classification target, compute `df["target"].value_counts()` and plot a bar chart. Compute the imbalance ratio (majority class count divided by minority class count). If the ratio exceeds 5:1, flag this clearly — it will affect your evaluation strategy in Phase 7.

Phase 2: Exploratory Data Analysis | CLO 1, CLO 3 | 5 marks

EDA is the most important phase of any data science project. Its purpose is to build a genuine intuitive understanding of every feature and every relationship in the dataset before any model is trained. Never rush it.

- 2.1. **Univariate distributions.** For every numerical feature, plot a histogram with KDE overlay (`seaborn.histplot(kde=True)`). Compute skewness and excess kurtosis using

`scipy.stats.skew` and `scipy.stats.kurtosis`:

$$\gamma_1 = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3}, \quad \gamma_2 = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4} - 3$$

Write a markdown interpretation for each feature: is the distribution symmetric, right-skewed ($\gamma_1 > 1$), or left-skewed ($\gamma_1 < -1$)? Does it show heavy tails ($\gamma_2 > 1$) that could make the mean misleading?

2.2. Categorical analysis. For each categorical feature, produce a bar chart of value counts sorted by frequency. Identify rare categories (those comprising less than 1% of the data) that may need grouping or special handling during encoding.

2.3. Correlation matrix. Compute the Pearson correlation matrix for all numerical features and visualise it as an annotated heatmap using `seaborn.heatmap` with `annot=True` and `cmap="coolwarm"`. Mask the upper triangle. The Pearson coefficient:

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Identify the 3 strongest positive and 3 strongest negative correlations with your regression target. Discuss any feature pair with $|r| > 0.8$ — multicollinearity at this level can destabilise regression coefficients.

2.4. Bivariate plots. Create:

- Scatter plots of the top-5 features vs. the regression target, each with a `np.polyfit(deg=1)` trend line overlaid.
- Box plots of each numerical feature grouped by the classification target class — this directly shows how well each feature separates the classes.
- A pair plot (`seaborn.pairplot`) of the 5 most correlated features, coloured by the classification target label. (For very large datasets, you may sample up to 2000 rows for the pair plot.)

2.5. Temporal or spatial patterns (if applicable). If the dataset has a timestamp column, plot a time-series line chart of the target over time and identify any trends, seasonality, or anomalies. If it has a location column (city, region, country), produce a grouped bar chart comparing the target across locations. If neither applies, substitute a violin plot comparing two of your most interesting features stratified by class.

2.6. EDA summary narrative. Write a **200–300 word markdown cell** summarising the 5 most important findings from your exploration. Every finding must be supported by a cited statistic or figure reference (e.g. “Feature X shows a strong positive correlation with the target, $r = 0.82$, visible in Figure 3, suggesting that higher X reliably predicts higher output”).

Phase 3: Data Cleaning & Preprocessing | CLO 1 | 4 marks

Document every cleaning decision as you make it. An undocumented cleaning step is a reproducibility failure. A future colleague (or your future self) must be able to read your notebook and understand exactly what was done and why.

3.1. Missing value audit. Compute missing counts and percentages per column. Visualise with `seaborn.heatmap(df.isnull())` or a sorted bar chart of missing percentages. For each column with missing values, record your decision in a markdown table:

Column	Strategy	Justification
<i>e.g. Air_Temp</i>	Median imputation	Right-skewed distribution; mean is inflated by extremes
...	...	...

Standard strategies: drop the column ($> 40\%$ missing), drop the row ($< 1\%$ of rows affected), impute with median (skewed numeric), impute with mean (approximately normal numeric), impute with mode (categorical), or forward-fill (time-series).

3.2. Outlier detection and treatment. Apply the IQR fences to each numerical feature:

$$L = Q_1 - 1.5 \cdot \text{IQR}, \quad U = Q_3 + 1.5 \cdot \text{IQR}, \quad \text{IQR} = Q_3 - Q_1$$

Visualise with box plots *before and after* treatment, arranged in a single subplot grid. For each affected feature, choose one action and justify it: remove the row, cap at the fence values (Winsorisation), or retain (the extreme value is physically valid and informative).

3.3. Categorical encoding. Apply appropriate encoding to every categorical feature and document each choice in a markdown table:

- *One-hot encoding* — nominal features with ≤ 8 categories
- *Ordinal encoding* — ordered categories (e.g. Low / Medium / High)
- *Frequency encoding* — high-cardinality nominal features (> 8 categories)

3.4. Data quality report. After all cleaning steps, print the final shape, confirm zero missing values with `assert df.isnull().sum().sum() == 0`, and display a `df.describe()` of the cleaned data. State the number of rows removed, columns dropped, and new columns created.

3.5. Train/test split. Split the cleaned dataset into 80% training and 20% test sets. Fit all scalers and encoders on training data only:

```

1 from sklearn.model_selection import train_test_split
2
3 X = df_clean.drop(columns=["reg_target", "cls_target"])
4 y_reg = df_clean["reg_target"]
5 y_cls = df_clean["cls_target"]
6
7 X_train, X_test, y_train, y_test, y_train, y_test = train_test_split(
8     X, y_reg, y_cls,
9     test_size=0.2, random_state=42,
10    stratify=y_cls # preserve class proportions in each split
11 )
12 print(f"Train: {X_train.shape} | Test: {X_test.shape}")

```

Phase 4: Data Visualization & Storytelling | CLO 3 | 4 marks

Visualisation is communication. Every plot must have a purpose you can state in one sentence. A beautiful plot with no insight is worth nothing; an ugly plot with a genuine insight is worth more.

- 4.1. Distribution dashboard.** Use `plt.subplot()` to create a 2×3 grid showing KDE-overlaid histograms for the 6 most informative numerical features. Title each subplot with the feature name and skewness value, e.g. “Air Pressure ($\gamma_1 = -0.43$)”.
- 4.2. Annotated correlation heatmap.** Full annotated heatmap with `cmap="coolwarm"` and upper-triangle masked. Use `ax.annotate()` to label the three cell pairs with the highest $|r|$ directly on the heatmap.
- 4.3. Target analysis plot.**
- *Regression:* scatter plot of the single best predictor vs. the regression target with a fitted line and an inset text box showing the simple linear R^2 and RMSE.
 - *Classification:* a 1×3 subplot row of violin plots — one subplot per top-3 discriminating feature, each violin split by class label.
- 4.4. Feature–target correlation bar chart.** Horizontal bar chart of Pearson correlations with the regression target, sorted by absolute value. Colour bars by sign: positive correlations in blue, negative in red. Draw a vertical line at $x = 0$.
- 4.5. Joint density plot.** A 2-D hexbin or KDE contour plot of the two most correlated features, with marginal histograms placed on both axes using `plt.subplot()` with manually positioned axes and shared tick ranges.
- 4.6. Domain-specific insight plot.** One custom plot revealing an insight specific to your dataset’s domain. Examples: a time-series line with anomaly event markers; a grouped bar chart by city (Lahore, Karachi, Islamabad, Faisalabad) comparing a key metric; a phase-space diagram for a sensor dataset; a class-stratified ECDF. In a markdown cell immediately after, write 3–5 sentences explaining what the plot reveals and why it matters.
- 4.7. Visualization standards.** Every plot in this section must satisfy all of:
- Descriptive title that explains what the plot shows
 - Axis labels with units in parentheses: e.g. “Pressure (kPa)”
 - Legend if multiple series or classes are plotted
 - Minimum figure size of 8×5 inches
 - `plt.tight_layout()` before `plt.show()`
 - A markdown caption immediately below explaining the key takeaway

Phase 5: Feature Engineering | CLO 1 | 3 marks

Raw features are rarely optimal inputs for a model. Feature engineering bridges the gap between what the dataset provides and what the model needs to find patterns.

- 5.1. Domain-informed new features.** Create at least **3 new features** motivated by domain knowledge of your dataset — not arbitrary mathematical combinations. For each, write a markdown cell explaining the physical or statistical motivation. Examples:
- Ratio of two related quantities (e.g. power divided by speed gives torque)
 - Interaction term capturing a known joint effect (e.g. temperature $\times$ pressure)
 - Log-transform of a right-skewed feature to reduce the influence of extreme values
 - Rolling mean or lag feature for time-series datasets
 - Polynomial term for a feature with a known nonlinear relationship to the target
- 5.2. Standardisation.** Fit `StandardScaler` on the training set only, then transform both splits. Do not fit on the full dataset or on the test set:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Fitting on test data leaks test-set statistics into the model, invalidating the evaluation.

5.3. Feature selection: two methods. Apply both methods and display their top-10 feature rankings side by side as a horizontal double bar chart:

- *Filter method:* absolute Pearson/Spearman correlation with the regression target (already computed in Phase 2 — re-use).
- *Embedded method:* Lasso (L_1) coefficient magnitudes. Lasso adds a penalty on the absolute sum of coefficients θ , driving irrelevant features to exactly zero:

$$J_{\text{Lasso}}(\theta) = \|\mathbf{y} - \mathbf{X}\theta\|_2^2 + \alpha\|\theta\|_1$$

The same α notation is used throughout: Ridge uses $\|\theta\|_2^2$, Lasso uses $\|\theta\|_1$. Fit with `sklearn.linear_model.LassoCV` to select α by cross-validation.

In a markdown cell, discuss: do both methods agree on the top features? Where they disagree, which ranking do you find more trustworthy for your dataset and why?

5.4. Scikit-learn Pipeline. Assemble a `sklearn.pipeline.Pipeline` chaining imputer, scaler, and model estimator so that all preprocessing is fitted on training data automatically:

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.impute import SimpleImputer
4 from sklearn.linear_model import Ridge
5
6 pipe = Pipeline([
7     ("imputer", SimpleImputer(strategy="median")),
8     ("scaler", StandardScaler()),
9     ("model", Ridge()),
10 ])
11 pipe.fit(X_train, yr_train)
12 print("Test R2:", pipe.score(X_test, yr_test))
```

Phase 6: Regression Modelling | CLO 2, CLO 3 | 6 marks

Your goal is to build models that predict the continuous target variable, evaluate them rigorously, and interpret what they have learned about the data.

6.1. Baseline. Compute the trivial baseline — predict the training-set mean $\bar{y}_{\text{train}}$ for every test sample. This is the performance floor your models must beat. Compute and print its RMSE and R^2 (the latter is 0 by definition for this predictor).

6.2. Linear Regression. Fit `sklearn.linear_model.LinearRegression` and evaluate on the test set using all three metrics:

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}, \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

6.3. Ridge Regression. Fit `sklearn.linear_model.RidgeCV` with a logarithmic grid of α values. Plot R^2 vs. $\log_{10}(\alpha)$ to visualise the regularisation path and show where the optimal α lies. The Ridge objective penalises large coefficients:

$$J_{\text{Ridge}}(\theta) = \|\mathbf{y} - \mathbf{X}\theta\|_2^2 + \alpha\|\theta\|_2^2$$

6.4. Residual analysis. For your best-performing model, produce:

- A scatter plot of residuals vs. fitted values. A truly good fit shows no pattern here; a funnel shape indicates heteroscedasticity; a curve indicates a missing nonlinear term.
- A Q-Q plot of residuals using `scipy.stats.probplot` — points following the diagonal confirm approximate normality.
- Identify the 5 samples with the highest absolute residuals. Inspect their raw feature values and hypothesise in 3–4 sentences why the model failed on these specific samples.

6.5. Model comparison table. Fill in your actual numeric results:

Model	R^2	RMSE	MAE
Baseline (mean predictor)	0.000	—	—
Linear Regression	—	—	—
Ridge Regression	—	—	—

6.6. Coefficient interpretation. Write a **150–200 word markdown** interpreting your best model's top-10 coefficients by magnitude. For each: does the direction (positive or negative) make physical sense? Is the magnitude surprising? What does a one-standard-deviation increase in this feature do to the predicted output?

Phase 7: Classification Modelling & Evaluation | CLO 2, CLO 3 | 6 marks

Classification requires a fundamentally different evaluation mindset from regression. Accuracy alone is unreliable — especially with imbalanced classes — and the choice of threshold has direct consequences for the real-world cost of errors.

7.1. Logistic Regression. Fit `sklearn.linear_model.LogisticRegression` with `max_iter=1000`. For multi-class targets, use `multi_class="ovr"` (one-vs-rest). Following the notation from Lab 10, the binary model maps a linear score z to a predicted probability $\hat{p}$:

$$z = \boldsymbol{\theta}^T \mathbf{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d, \quad \hat{p} = P(Y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$\boldsymbol{\theta}$ is the coefficient vector (including intercept θ_0) fitted by minimising cross-entropy. OvR trains K binary classifiers; class k is treated as positive and all others as negative. Final prediction is the class with the highest $\hat{p}$.

7.2. Confusion matrix. Plot the confusion matrix as an annotated heatmap with class labels on both axes. Each cell must show both the raw count and the percentage of total test samples. This makes it immediately clear which type of error dominates.

7.3. Classification report. Use `sklearn.metrics.classification_report` to print per-class precision, recall, and F1:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

In a markdown cell, interpret the per-class values: which class is hardest to predict? What type of error — false positive or false negative — is most costly in your application domain, and does your model's error distribution reflect that priority?

7.4. ROC–AUC and PR curves.

- *Binary classification:* plot the ROC curve and annotate with AUC. On a second set of axes, plot the Precision–Recall curve.

- **Multi-class:** plot one-vs-rest ROC curves for each class on the same axes, each with its own AUC annotation.

Write a markdown cell explaining why the PR curve is more informative than the ROC curve when classes are severely imbalanced.

- 7.5. Threshold tuning** (binary targets only). Compute precision, recall, and F1 for decision thresholds from 0.1 to 0.9 in steps of 0.05. Plot all three metrics vs. threshold on the same axes. Mark the threshold that maximises F1. Explain in 3–4 sentences why the default threshold of 0.5 may not be optimal for your specific application (consider the cost asymmetry between false positives and false negatives in your domain).
- 7.6. Second classifier.** Train one additional classifier of your choice (e.g. `RandomForestClassifier` or `KNeighborsClassifier`) and compare it with Logistic Regression:

Model	Accuracy	Macro F1	ROC-AUC	Fit time (s)
Logistic Regression	—	—	—	—
<i>Your second model</i>	—	—	—	—

- 7.7. Error analysis.** Identify 10 misclassified test samples. Inspect their raw feature values and the model’s predicted probabilities. Write 100–150 words hypothesising why the model failed on these specific samples and what additional features or data would most likely improve performance.

Phase 8: Unsupervised Analysis: K-Means Clustering | CLO 2 | 5 marks

Note: K-Means clustering is covered in an **upcoming lab session**. Study the algorithm using the lab manual and the resources listed at the end of this document before completing this phase. Treat it as an opportunity to apply a new technique independently — a core professional skill.

Clustering asks a different question from classification: rather than predicting a known label, it asks *what natural groupings exist in the data?* The answer may confirm, challenge, or add nuance to what the supervised models found.

- 8.1. Algorithm and theory.** In a markdown cell, describe the K-Means algorithm step by step and state the objective it minimises:

$$\text{Inertia} = \sum_{j=1}^k \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2$$

where $\boldsymbol{\mu}_j$ is the centroid of cluster C_j . Discuss two known limitations of K-Means (e.g. sensitivity to initialisation, assumption of convex clusters, difficulty with varying cluster densities).

- 8.2. Optimal k .** Run `sklearn.cluster.KMeans` for $k = 2, 3, \dots, 10$ with `n_init=10`, `random_state=42`. Plot the elbow curve (WCSS vs. k) and the mean silhouette score vs. k on the same figure using two y -axes. Justify your chosen k using both criteria. The silhouette score for sample i :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ is the mean distance from sample i to all other points in its assigned cluster, and $b(i)$ is the mean distance from sample i to all points in the nearest other cluster. A score near +1 means the sample fits its cluster well; near 0 means it lies on a cluster boundary.

- 8.3. Cluster visualisation.** Apply PCA to reduce the scaled feature matrix to 2 dimensions. Scatter-plot all test samples coloured by cluster label. Overlay the cluster centroids (projected into PCA space) as large star markers with a distinct edge colour. Add a legend identifying each cluster.
- 8.4. Cluster profiling.** For each cluster, compute the mean of every original (unscaled) feature. Plot a grouped bar chart comparing all cluster means. Assign a short descriptive label to each cluster based on the mean values, e.g. “High-temperature, low-speed operating regime” or “Low-load, stable-voltage condition”.
- 8.5. Cluster–target cross-tabulation.** Compute `pd.crosstab(cluster_labels, yc_test)` and display it as a stacked bar chart. Write a markdown paragraph discussing whether the discovered clusters align with the known class labels. If they do align well, what does this suggest about the feature space? If they do not, what might explain the discrepancy?

Phase 9: Streamlit Interactive Dashboard | CLO 4 | 15 marks

The dashboard is your deliverable product. A non-technical user — someone who has never seen your notebook — must be able to open it, understand the dataset, run predictions, and explore the clusters without any guidance from you. Design for that user, not for yourself.

9.1. Tab A — Dataset Overview.

- `st.dataframe()` with the cleaned dataset (built-in sort/filter)
- A metrics row using `st.metric`: number of rows, number of features, number of missing values (should be zero)
- `st.selectbox` to choose any column; dynamically render that column’s histogram below using `st.pyplot()`
- Full correlation heatmap displayed as a static Matplotlib figure via `st.pyplot()`

9.2. Tab B — Exploratory Analysis.

- Two `st.selectbox` widgets to choose X and Y features; scatter plot updates on every change
- `st.checkbox("Show trend line")` to toggle a linear fit on/off
- `st.selectbox("Colour by")` for a categorical feature to colour the points by class
- `st.slider` to filter the data to a numeric range of a chosen feature before plotting — so the user can zoom into a subpopulation

9.3. Tab C — Model Prediction.

- Load both saved models with `joblib.load()` decorated with `@st.cache_resource` — do **not** retrain inside the app
- Render one `st.number_input` or `st.slider` per input feature with sensible min/max derived from the training set
- On `st.button("Predict")`: simultaneously display the regression prediction and the classification prediction
- For classification: show predicted class probabilities as a bar chart (`st.bar_chart`) so the user sees the model’s confidence
- Colour-code the result with `st.success()` / `st.warning()` / `st.error()` according to the severity or risk level implied by the predicted value

9.4. Tab D — Cluster Explorer.

- PCA scatter plot of all test samples coloured by cluster label, with centroids overlaid, rendered via `st.pyplot()`

- `st.selectbox` to choose a cluster to highlight; re-render the scatter plot with the selected cluster in full colour and all other clusters greyed out
- A table comparing mean feature values of the selected cluster against the dataset overall (`st.dataframe`)
- A bar chart showing the delta between selected-cluster means and overall means, so the user can immediately see what makes that cluster distinctive

9.5. Technical requirements.

- `@st.cache_data` on all data-loading and preprocessing functions
- `@st.cache_resource` on both model-loading functions
- `requirements.txt` with pinned versions of all packages
- `README.md` covering: install instructions (`pip install -r requirements.txt`), how to run (`streamlit run app.py`), and a one-paragraph description of each tab
- App must launch and run without errors from a fresh install on the instructor's machine

9.6. UX quality.

- Every widget has a human-readable label — not a raw column name
- Every plot has a title, axis labels, and a `st.caption()` sentence explaining what the user should notice
- Sidebar or page header clearly states the project title and dataset name
- Guard against bad user input: if the same feature is selected for both X and Y in Tab B, show `st.warning("Please select two different features.")` and suppress the plot

Deliverables

Submit **one ZIP archive** with the following exact internal structure:

```
FinalProject-2023-MC-XX_MC-YY/
|-- notebook/
|   '-- FinalProject-2023-MC-XX_MC-YY.ipynb      # fully executed, all outputs
|       visible
|-- dashboard/
|   |-- app.py                                  # Streamlit app (single file or multi-
|       page)
|   |-- pages/                                  # (only if using multi-page layout)
|   |-- models/
|       |-- regression_model.pkl
|       |-- classification_model.pkl
|   |-- data/
|       |-- cleaned_data.csv
|   '-- requirements.txt                        # all Python dependencies with pinned
|       versions
'-- README.md                                  # setup and run instructions
```

Notebook requirements:

- All 9 phases present with section headers matching the phase numbering
 - Fully executed: run *Kernel* → *Restart & Run All* before submitting
 - All code cell outputs visible — no empty `In [ ]` counters on teaching/demo cells
 - Markdown cells with theory and interpretation between every pair of code cells
 - No placeholder code (`# YOUR CODE HERE`) anywhere in the submitted notebook — every cell must be fully implemented
 - All $\text{\LaTeX}$ equations render correctly in Jupyter
-

Grading Rubric (Total: 50 marks)

Phase	Assessment Criteria	CLO	Marks
Data Acquisition	Dataset meets all size and diversity requirements; data card written; regression and classification targets defined and justified with domain reasoning; class imbalance ratio computed and interpreted.	CLO 1	2
Exploratory Analysis	Univariate distributions with skewness and kurtosis computed and interpreted for all numerical features; correlation heatmap produced and top pairs discussed including multicollinearity concern; bivariate scatter, box, and pair plots present; EDA summary narrative is 200–300 words with 5 findings each citing a statistic.	CLO 1, 3	5
Data Cleaning	Missing values audited with a documented decision table; outlier detection via IQR with before/after box plots and per-feature justification; categorical encoding applied with strategy documented; train/test split stratified; data quality report confirms zero missing values.	CLO 1	4
Visualization	All 6 required plots present (distribution dashboard, annotated heatmap, target analysis, correlation bars, joint density, domain insight); all plots satisfy the checklist (title, axis labels with units, legend, caption); domain insight plot reveals a genuine finding with a written explanation.	CLO 3	4
Feature Eng.	At least 3 domain-motivated new features created with written justification for each; two feature selection methods applied and top-10 rankings compared; scaler fitted only on training data; Pipeline assembled and demonstrated.	CLO 1	3
Regression	Baseline computed; Linear and Ridge models evaluated on test set with R^2 , RMSE, MAE; regularisation path plotted; residual analysis includes residual-vs-fitted and Q-Q plots with high-residual sample inspection; comparison table filled with actual values; coefficient interpretation is 150–200 words and physically grounded.	CLO 2, 3	6

Phase	Assessment Criteria	CLO	Marks
Classification	Logistic regression trained and evaluated; confusion matrix shows counts and percentages; classification report interpreted per class; ROC-AUC and PR curves produced; threshold tuning performed for binary targets; second classifier trained and compared in a table; error analysis is 100–150 words with a hypothesis.	CLO 2, 3	6
Clustering	K-Means algorithm and WCSS objective explained with equation and limitations noted; elbow and silhouette plots produced; optimal k justified with both criteria; PCA scatter with centroids displayed; cluster profiles have descriptive labels; cross-tabulation vs. classification target discussed.	CLO 2	5
Dashboard	All 4 tabs functional and correct; models loaded from file (not retrained inside app); caching decorators applied to data and model loading; bad-input error state handled with <code>st.warning</code> ; <code>requirements.txt</code> and <code>README.md</code> present; app runs clean from a fresh install; UX polished with proper labels, captions, and sidebar branding.	CLO 4	15
Total			50

Note: Viva note. During the demo, the instructor will question **both group members independently** on any plot, metric, or modelling decision in the notebook, and will ask one or both to navigate the dashboard live. If either member cannot explain the group's work, up to 20% may be deducted from the marks of the relevant phase *for both partners*. A member who cannot explain what $R^2 = 0.87$ means, why Ridge was chosen over plain Linear Regression, or how the silhouette score guided the choice of k will cause both partners to lose marks.

Academic Integrity

- This is a **group project of two students**. Both partners are expected to contribute equally to all phases. Sharing code, analysis, or interpretation text *between groups* is academic dishonesty.
- AI tools may be used for syntax assistance, debugging, documentation lookup, and boilerplate code generation. However, all interpretations, explanations, captions, conclusions, and decision justifications submitted in the notebook **must be written and understood by the students themselves**.
- All code must use Python libraries taught in this course: NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, SciPy, and Streamlit. External ML libraries (TensorFlow, PyTorch, XGBoost) require prior written approval.
- Similarity checks will be run across all submitted notebooks and dashboard files.
- Submitting a project you cannot explain during the viva constitutes academic dishonesty regardless of whether the code runs correctly.

Resources

Course Lab Manuals (LMS)	Review Labs 05–13 for Pandas, cleaning, modelling, and evaluation
Scikit-learn User Guide	https://scikit-learn.org/stable/user_guide.html
Streamlit Documentation	https://docs.streamlit.io
Pandas User Guide	https://pandas.pydata.org/docs/user_guide/index.html
Data 100 Course Notes (UC Berkeley)	https://ds100.org/fa25/ — EDA, regression, and classification
StatQuest with Josh Starmer	https://www.youtube.com/@statquest — logistic regression, clustering
Kaggle Learn: Intermediate ML	https://www.kaggle.com/learn/intermediate-machine-learning

— End of Final Project Specification —

Project Groups

Groups are fixed as listed below. Submit **one ZIP archive per group** using the naming convention shown on page 1. Both members of each group receive the same grade for all phases. Each member must be present at the viva and must be able to explain any part of the submitted work independently.

ZIP filename format: FinalProject-2023-MC-XX_MC-YY.zip where XX is the lower and YY is the higher roll number in your group (e.g. FinalProject-2023-MC-22_MC-28.zip).

Group	Member 1	Member 2
1	2023-MC-22	2023-MC-28
2	2023-MC-08	2023-MC-32
3	2023-MC-16	2023-MC-54
4	2023-MC-13	2023-MC-55
5	2023-MC-45	2023-MC-68
6	2023-MC-67	2023-MC-75
7	2023-MC-14	2023-MC-27
8	2023-MC-02	2023-MC-61
9	2023-MC-31	2023-MC-44
10	2023-MC-07	2023-MC-09
11	2023-MC-41	2023-MC-72
12	2023-MC-18	2023-MC-59
13	2023-MC-15	2023-MC-42
14	2023-MC-71	2023-MC-73
15	2023-MC-01	2023-MC-36
16	2023-MC-29	2023-MC-60
17	2023-MC-56	2023-MC-62
18	2023-MC-53	2023-MC-65
19	2023-MC-20	2023-MC-70
20	2023-MC-43	2023-MC-50
21	2023-MC-51	2023-MC-63
22	2023-MC-24	2023-MC-30
23	2023-MC-06	2023-MC-17
24	2023-MC-26	2023-MC-33
25	2023-MC-57	2023-MC-66
26	2023-MC-05	2023-MC-52
27	2023-MC-19	2023-MC-46
28	2023-MC-03	2023-MC-04
29	2023-MC-10	2023-MC-48

(continued...)

Group	Member 1	Member 2
30	2023-MC-23	2023-MC-49